

14-AI-TESTING-STANDARDS.md

Version : V1.0

Statut : ACTIVE

Catégorie : AI Engineering Framework

Projet : Vitala

1. Objectif

AI-TESTING-STANDARDS.md définit les règles obligatoires de tests applicables à toutes les fonctionnalités développées sur Vitala.

Il garantit que chaque modification :

- fonctionne correctement ;
 - ne casse pas l'existant ;
 - respecte la sécurité ;
 - reste maintenable.
-

2. Principe fondamental

Une fonctionnalité sans test approprié est considérée comme incomplète.

L'IA ne doit jamais déclarer une tâche terminée uniquement parce que :

- le code compile ;
 - l'écran s'affiche ;
 - une démonstration manuelle fonctionne.
-

3. Cycle obligatoire de validation

Toute fonctionnalité doit suivre :

```text id="q1m7np" Développement

↓

Test automatique

↓

Correction des erreurs

↓

Validation fonctionnelle

↓

Vérification sécurité

↓

Documentation

↓

Livraison

---

#### # 4. Types de tests obligatoires

Selon le contexte, l'IA doit utiliser :

##### ## 4.1 Tests unitaires

Objectif :

Tester une petite unité indépendante.

Exemples :

- fonction ;
- service ;
- utilitaire ;
- logique métier.

---

##### ## 4.2 Tests composants

Objectif :

Vérifier les composants interface.

Exemples :

- affichage ;
- interactions ;
- états ;
- erreurs.

---

## ## 4.3 Tests intégration

Objectif :

Vérifier que plusieurs parties fonctionnent ensemble.

Exemples :

- frontend + API ;
- service + database ;
- auth + permissions.

---

## ## 4.4 Tests end-to-end

Objectif :

Tester un parcours utilisateur complet.

Exemples :

- inscription ;
- création d'un Flash ;
- synchronisation ;
- paiement ;
- administration.

---

## # 5. Règles de tests Frontend

L'IA doit tester :

- rendu correct ;
- interactions utilisateur ;
- états de chargement ;
- états d'erreur ;
- permissions ;
- responsive.

Une interface doit gérer :

```text id="nqv7to"

Chargement

↓

Succès

↓

Erreur

↓

Données vides

↓

Absence de permission

6. Règles de tests Backend

L'IA doit vérifier :

- logique métier ;
- validation ;
- erreurs ;
- sécurité ;
- performances.

Chaque service important doit avoir une couverture adaptée.

7. Règles de tests API

Chaque endpoint doit être testé avec :

Cas valide

Exemple :

Utilisateur autorisé effectue une action.

Cas invalide

Exemple :

Données incorrectes.

Cas non autorisé

Exemple :

Utilisateur sans permission.

Cas limite

Exemple :

Données extrêmes.

8. Tests Database

Toute modification Database doit vérifier :

- migration correcte ;
- contraintes ;
- relations ;
- permissions ;
- RLS.

Cas obligatoires :

```text id="1v0n8v" Utilisateur autorisé

↓

Accès accepté

---

Utilisateur non autorisé

↓

Accès refusé

---

## # 9. Tests RLS

Chaque nouvelle politique RLS doit vérifier :

- qui peut lire ;
- qui peut écrire ;
- qui peut modifier ;
- qui peut supprimer.

Une politique trop permissive est considérée comme un échec.

---

#### # 10. Tests Offline First

Pour les fonctionnalités offline, l'IA doit tester :

- création hors connexion ;
- stockage local ;
- reprise réseau ;
- synchronisation ;
- conflits ;
- récupération après erreur.

---

#### # 11. Tests Synchronisation

Le système Sync doit vérifier :

- opérations en attente ;
- ordre des événements ;
- doublons ;
- conflits ;
- récupération.

---

#### # 12. Tests Sécurité

Avant livraison :

L'IA doit vérifier :

- authentification ;
- permissions ;
- données sensibles ;
- secrets ;
- injections ;
- accès non autorisés.

---

#### # 13. Tests de performance

Lorsque nécessaire :

Tester :

- temps de réponse ;
- taille des données ;
- consommation mémoire ;
- nombre de requêtes.

---

#### # 14. Tests de régression

Toute correction importante doit vérifier que :

- les anciennes fonctionnalités fonctionnent encore ;
- les modules existants ne sont pas cassés.

---

#### # 15. Tests liés à l'IA

Les fonctionnalités utilisant l'IA doivent tester :

- entrée correcte ;
- sortie attendue ;
- cas inattendus ;
- limites ;
- erreurs modèle.

---

#### # 16. Règle de couverture

L'objectif n'est pas uniquement un pourcentage.

La priorité est :

- tester les parties critiques ;
- tester les règles métier ;
- tester les zones sensibles.

---

#### # 17. Gestion d'un test échoué

Si un test échoue :

L'IA doit :

1. identifier la cause ;
2. corriger ;
3. relancer les tests ;
4. documenter si nécessaire.

Elle ne doit jamais supprimer un test uniquement pour obtenir un résultat positif.

---

#### # 18. Rapport de validation obligatoire

À la fin d'une tâche, l'IA doit fournir :

```text id="0kq5fa"

Tests réalisés :

- Test 1 :

Résultat :

- Test 2 :

Résultat :

Problèmes rencontrés :

Corrections effectuées :

19. Checklist avant livraison

☐ Tests unitaires réalisés

☐ Tests intégration réalisés

☐ Tests sécurité réalisés

☐ Tests erreurs réalisés

☐ Tests régression vérifiés

☐ Documentation mise à jour

20. Règle finale

Un code non testé est un risque.

Dans Vitala :

Fonctionnalité créée + tests validés = fonctionnalité terminée.

Historique

| Version | Modification |
|---------|------------------------------------|
| V1.0 | Création des standards de tests IA |

Fin de AI/14-AI-TESTING-STANDARDS.md